

Conceitos básicos

A performance no React refere-se à eficiência e rapidez com que um aplicativo React é executado. Ela abrange vários aspectos, incluindo o tempo de carregamento inicial, a velocidade de renderização dos componentes, a fluidez da interação do usuário e a utilização eficiente dos recursos do dispositivo.

Uma aplicação React com bom desempenho é capaz de atualizar a interface do usuário de forma eficiente, responder rapidamente às interações do usuário e evitar atrasos perceptíveis. Portanto, otimizar a performance no React é crucial para garantir uma experiência de usuário positiva.

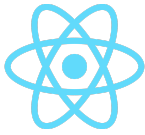
Vamos conhecer alguns conceitos básicos que envolvem performance no React.

Virtual DOM

O [Virtual DOM](#) (DOM virtual) é um conceito utilizado pelo React para melhorar a eficiência na atualização da interface do usuário.

No React, em vez de atualizar diretamente o DOM real sempre que ocorrem alterações nos dados ou nas propriedades dos componentes, o React cria uma cópia em memória chamada Virtual DOM. Quando há uma atualização no estado ou nas propriedades de um componente, o React compara a representação virtual anterior (Virtual DOM) com a nova representação virtual. Ele identifica as diferenças entre as duas representações e, em seguida, realiza apenas as alterações necessárias no DOM real.

Assim o React consegue otimizar o processo, aplicando as mínimas alterações necessárias, evitando atualizações desnecessárias e melhorando a experiência do usuário.



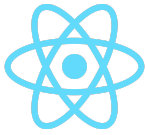
Renderização Condicional

A [renderização condicional](#) no React permite decidir se um componente deve ser renderizado com base em determinadas condições. Isso significa que você pode controlar dinamicamente o que é exibido na interface do usuário com base em lógica ou estados específicos.

Aqui está um exemplo simples de renderização condicional no React:

```
function App() {  
  const isAuthenticated = true;  
  
  return (  
    <div>  
      {isAuthenticated ? <UserDashboard /> : <LoginScreen />}  
    </div>  
  );  
}  
  
function UserDashboard() {  
  return <h1>Bem-vindo! </h1>;  
}  
  
function LoginScreen() {  
  return <h1>Faça o seu login</h1>;  
}
```

Nesse exemplo, o componente **UserDashboard** só será renderizado se a variável `isAuthenticated` for `true`.



Imutabilidade

A [imutabilidade](#) no React se refere ao conceito de que os dados no aplicativo devem ser tratados como "imutáveis", ou seja, eles não devem ser modificados diretamente. Em vez disso, sempre que um dado precisa ser alterado, uma nova versão desse dado é criada.

Isso significa que, em vez de modificar um objeto existente, você cria um novo objeto com as alterações desejadas. Isso é importante porque evita efeitos colaterais inesperados e torna mais fácil rastrear as mudanças nos dados.

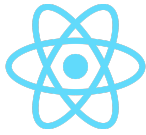
Exemplo:

```
function MyComponent() {
  const [counter, setCounter] = useState(0);

  const handleClick = () => {
    setCounter(prevCounter => prevCounter + 1);
  };

  return (
    <div>
      <h1>{counter}</h1>
      <button onClick={handleClick}>Incrementar</button>
    </div>
  );
}
```

Neste exemplo, usamos o hook `useState` para criar um estado chamado `counter`. Para atualizar o valor do contador, usamos a função `setCounter`, passando o estado anterior como argumento e retornando o novo valor. Isso garante a imutabilidade do estado.



Otimização de listas

Ao renderizar listas de elementos dinâmicos, é fundamental utilizar chaves únicas ([key](#)) para identificar cada elemento da lista. Além disso, evite realizar operações complexas ou custosas dentro do loop de renderização ao usar o método `map`. Essas operações podem prejudicar o desempenho do aplicativo. Exemplo de uso de chaves únicas:

```
<ul>
  {lista.map(item => (
    <li key={item.id}>{item.text}</li>
  ))}
</ul>
```

Além disso, a [memoização](#) em componentes de lista também desempenha um papel crucial na melhoria da performance ao evitar re-renderizações desnecessárias de itens individuais da lista.